

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 02

Financial Data and Returns

Version 2

Start with a stock-price record. Identify what it measures, calculate a return, and explain what that return leaves out.

Most examples reuse Microsoft data from August 14-21, 2024. Apple's split and Netflix's price decline answer two different questions that this short Microsoft sample cannot answer.

Issue	Question	Page
01	What data did we download?	2
02	What do the price columns mean?	3
03	How much did the closing price change?	4
04	Does the return include the dividend?	5
05	Does a stock split make an investor richer?	6
06	How much is the stock worth in TWD?	7
07	Why use log returns?	8
08	What is the return over several days?	9
09	Which average return should we use?	10
10	Is an absent date a missing price?	11
11	Should we delete a very large return?	12

Use the examples

Read the question, predict the named result, then inspect the code and output. The discussion questions are part of each issue; there is no separate practice section.

Run the code in order in a fresh Python notebook with numpy, pandas and yfinance installed. Outputs are shown here, so you can follow the calculations without running code during class.

Stock prices are Yahoo Finance observations retrieved through yfinance, not records of personal trades. Returns exclude trading fees and taxes. Dividends are included only where stated; no trading strategy is evaluated.

ISSUE 01

What data did we download?

First identify the company, field, currency and dates. MSFT is Microsoft common stock, listed on Nasdaq; its prices here are in USD per share.

The start date is included; the end date is excluded. A daily row labels a trading date, not a trade at midnight. This request uses daily data without pre-market or after-hours data.

Before running: Count the weekdays from August 14 through August 21. Will August 22 appear?

```
import numpy as np
import pandas as pd
import yfinance as yf

msft = yf.Ticker("MSFT").history(
    start="2024-08-14", end="2024-08-22",
    interval="1d", auto_adjust=False,
    actions=True, prepost=False, keepna=True,
)
msft.index = msft.index.strftime("%Y-%m-%d")
assert msft.index.is_unique
assert msft.index.is_monotonic_increasing
close = msft["Close"]
assert close.notna().all() and (close > 0).all()
print("Dates:", ", ", ".join(msft.index))
print("Rows:", len(msft))
```

Dates: 2024-08-14, 2024-08-15, 2024-08-16, 2024-08-19, 2024-08-20, 2024-08-21

Rows: 6

The request returns 6 rows, ending on August 21. August 17-18 are a weekend. Six closing prices give five adjacent close-to-close returns, because each return needs two prices.

Setting	Meaning
MSFT	Microsoft common stock
1d	One row per trading date
auto_adjust=False	Keep Close and Adj Close separate

The checks stop calculation if dates repeat or prices are missing. They do not prove that the provider's observations are correct.

Discuss

1. Why can this request produce five returns, not six? Identify the extra price needed to calculate a return ending on August 14.

2. Read the request: what would changing end to August 21 remove, and which last return would disappear?

Sources: [1] yfinance: historical-data parameters; [2] Microsoft: dividend announced June 12, 2024. See the linked references at the end.

ISSUE 02

What do the price columns mean?

Open, High, Low and Close describe different prices within a trading session. Choose the field before choosing the formula.

Field	Meaning
Open / Close	Opening / closing price
High / Low	Highest / lowest session price
Volume	Trading activity, not a price

Use O, H, L and C for these price fields, in USD per share. The session range is H minus L. The open-to-close change is C minus O: it can be negative even though the range is positive.

Before running: For August 15, will the session range be at least as large as the absolute open-to-close change?

```
day = msft.loc["2024-08-15"]
result = pd.DataFrame({
    "Measure": ["High - Low", "Close - Open"],
    "USD per share": [
        f"{day['High'] - day['Low']:.2f}",
        f"{day['Close'] - day['Open']:.2f}",
    ],
})
result
```

Measure	USD per share
High - Low	3.45
Close - Open	1.23

MSFT's range was USD 3.45 per share; its open-to-close increase was USD 1.23 per share. The first measures the day's range, not a return earned by buying and selling.

The daily bar does not reveal whether the high came before the low. It also does not establish the exact price at which your order would have traded.

Discuss

1. Using this daily bar, can you claim that buying at the low and selling later at the high was possible? Name the missing information.
2. To measure the overnight price change into August 15, which two fields and dates would you use instead of Close minus Open?

Sources: [1] yfinance: historical-data parameters; [10] Yahoo Finance: MSFT historical data. See the linked references at the end.

ISSUE 03

How much did the closing price change?

A dollar gain and a percentage return answer different questions. Divide the price change by the starting price to express the change relative to the initial value.

For MSFT, the downloaded closes are USD 416.86 per share on August 14 and USD 421.03 on August 15. Subtract the first from the second, then divide by the first.

C_{t-1} and C_t are successive closing prices in USD per share. r_t is the simple price return, stored as a decimal. Both prices must use the same share basis, and the starting price must be positive.

$$r_t = \frac{C_t - C_{t-1}}{C_{t-1}} = \frac{C_t}{C_{t-1}} - 1$$

Before running: Will the dollar increase and percentage increase have the same numerical value? Explain which starting value the percentage uses.

```
start_close = close.loc["2024-08-14"]
end_close = close.loc["2024-08-15"]
price_change = end_close - start_close
price_return = price_change / start_close
print(f"Price change: USD {price_change:.2f} per share")
print(f"Price return: {price_return:.2%}")
```

Price change: USD 4.17 per share

Price return: 1.00%

The price increased by USD 4.17 per share, or 1.00%. A 1% return is not a USD 1 profit: it is a gain equal to 1% of the starting value.

This calculation covers the August 14 close to the August 15 close. It excludes the dividend, fees and taxes; it is not a record of a particular investor's realized profit.

Discuss

1. Convert the displayed percentage back to a decimal. Which starting value must you multiply it by to recover the price change, and why might rounded numbers differ slightly?
2. The result is positive. What information is still needed before saying that a student who traded MSFT on August 15 made this return?

Sources: [7] Zivot: Return Calculations; [10] Yahoo Finance: MSFT historical data. See the linked references at the end.

ISSUE 04

Does the return include the dividend?

A price-only return leaves out cash distributions. Microsoft's actual August 2024 dividend lets us add one item to the previous calculation.

Dividend	MSFT announcement
Amount	USD 0.75 per share
Ex-dividend date	August 15, 2024
Payment date	September 12, 2024

For this regular dividend, a purchase before August 15 normally qualifies; a purchase on August 15 or later does not. An already eligible holder who sells on August 15 normally keeps the dividend entitlement. The payment date is when payment is due, not the deadline for buying.

D_t is the dividend per share in USD. For a holder eligible for this dividend, value the amount owed at face value; exclude taxes, fees and reinvestment. The dividend is not cash paid on August 15.

$$r_t^{\text{div}} = \frac{C_t + D_t - C_{t-1}}{C_{t-1}}$$

Before running: Does adding the USD 0.75 dividend increase the return by 0.75%, or by the dividend divided by the starting price?

```
dividend = msft.loc["2024-08-15", "Dividends"]
dividend_return = (end_close + dividend) / start_close - 1
result = pd.DataFrame({
    "Calculation": ["Price only", "Price + dividend"],
    "Return": [f"{price_return:.2%}", f"{dividend_return:.2%}"],
})
result
```

Calculation	Return
Price only	1.00%
Price + dividend	1.18%

Including the dividend changes the result from 1.00% to 1.18%. Adj Close also incorporates applicable distributions through price adjustments: do not add the dividend to its return again. Its ratio need not exactly equal this cash-dividend calculation.

Discuss

1. Which event date determines whether this dividend belongs in the holding-period calculation? Explain why the September payment date is not enough.
2. For August 14-15, choose Close or Adj Close for a price-only calculation. What is wrong with using an adjusted-price return and then adding the same dividend again?

Sources: [2] Microsoft: dividend announced June 12, 2024; [3] Investor.gov: ex-dividend dates; [10] Yahoo Finance: MSFT historical data. See the linked references at the end.

ISSUE 05

Does a stock split make an investor richer?

A split changes the number of shares and the price per share. It does not, by itself, create investment value.

Apple announced a four-for-one split in 2020. Split-adjusted trading began on August 31. The announcement grants three additional shares for each existing share; this is an actual corporate action, not an invented portfolio.

Apple split	Effect of the split alone
Share count	Multiplied by 4
Price per share	Divided by 4
Total position value	Unchanged

Let N be a share count and C a price in USD per share on the pre-split basis. Holding all other price changes aside, the same ownership value is represented after the split as follows.

$$(4N)\left(\frac{C}{4}\right) = NC$$

Before running: The downloaded history uses a common split-adjusted share basis. Should its August 28-31 price return mechanically be -75%?

```
apple = yf.Ticker("AAPL").history(
    start="2020-08-28", end="2020-09-01",
    auto_adjust=False, actions=True,
)
split = apple.loc["2020-08-31", "Stock Splits"]
split_day_return = apple["Close"].iloc[-1] / apple["Close"].iloc[0] - 1
print(f"Recorded split: {split:.0f}-for-1")
print(f"Close-series return, Aug 28-31: {split_day_return:.2%}")
```

```
Recorded split: 4-for-1
Close-series return, Aug 28-31: 3.39%
```

The split record is 4-for-1, while the downloaded Close series rises 3.39%. The split identity does not predict the actual market return. Do not divide this already split-adjusted history by four again.

Discuss

1. Why does a fourfold increase in shares not imply a fourfold increase in value? Use the split identity, not an assumed account balance.
2. Why would applying the split ratio again corrupt this return? Identify which input's share basis you would check before changing a price history.

Sources: [4] Apple: four-for-one split announcement; [1] yfinance: historical-data parameters; [10] Yahoo Finance: MSFT historical data. See the linked references at the end.

ISSUE 06

How much is the stock worth in TWD?

To express a USD price in New Taiwan dollars, multiply by the number of TWD per USD. Keep the quotation direction visible.

The Federal Reserve reports TWD 32.27 per USD for August 15, 2024. This is a New York noon reference observation, not an exchange rate synchronized with the stock close. Use it here only to translate that day's closing value, not to claim an executable trade.

C_t is the USD closing price per share. X_t is the reference rate in TWD per USD. V_t is the translated value in TWD per share.

$$V_t = C_t X_t$$

The USD units cancel: (USD per share) multiplied by (TWD per USD) gives TWD per share.

Before running: Will the numerical TWD value be larger or smaller than the USD price? Does that numerical change mean an investment gain?

```
# Federal Reserve H.10: observed Aug 15, released Aug 19, 2024.
twd_per_usd = 32.27
twd_value = end_close * twd_per_usd
print(f"Reference rate: {twd_per_usd:.2f} TWD per USD")
print(f"Translated close: TWD {twd_value:,.2f} per share")
```

```
Reference rate: 32.27 TWD per USD
Translated close: TWD 13,586.64 per share
```

The translated value is TWD 13,586.64 per share. It expresses the same stock-price observation in another currency; it does not show a new profit.

The H.10 page was released on August 19. It is suitable for this retrospective calculation, but that publication cannot be treated as information already available on August 15.

Discuss

1. Why is multiplication appropriate for a rate labeled TWD per USD? Show the cancellation of units.
2. Can this conversion establish what a broker would have paid in TWD at the August 15 close? Identify the observation-time and transaction information that is missing.

Sources: [5] Federal Reserve: H.10, August 19, 2024; [6] FRED: DEXTAUS units and observation time. See the linked references at the end.

ISSUE 07

Why use log returns?

Simple returns describe proportional gains. Log returns describe the same price ratios in a form that can be added over consecutive holding intervals.

For positive closing prices, let r_t be the simple return and ℓ_t the log return. $\ln$ is the natural logarithm; $\exp$ reverses it. Both are stored as decimals, not dollar amounts.

$$\ell_t = \ln\left(\frac{C_t}{C_{t-1}}\right) = \ln(1 + r_t), \quad r_t = e^{\ell_t} - 1$$

Before running: For the positive August 14-15 price return, will the log return be slightly smaller than the simple return?

```
log_return = np.log(end_close / start_close)
result = pd.DataFrame({
    "Return type": ["Simple", "Log"],
    "Value (%)": [f"{100*price_return:.4f}", f"{100*log_return:.4f}"],
})
result
```

Return type	Value (%)
Simple	1.0003
Log	0.9954

The simple return is 1.0003%; the log return is 0.9954%. They describe the same price ratio. Their difference is not a fee or a loss. Convert the log return back before calling it a percentage gain.

Why researchers use it

The logarithm turns a product of consecutive growth factors into a sum. Researchers can add daily log returns to form weekly or monthly log returns, and express return as a change in log price. This is useful algebra; it does not make returns independent or normally distributed.

Discuss

1. Recover the simple return with $\exp(\log_return) - 1$. Why does subtracting the two displayed returns fail to measure any money lost?
2. To summarize this price movement as a percentage gain, which value would you report? Explain one different task for which the log value is convenient.

Sources: [7] Zivot: Return Calculations. See the linked references at the end.

ISSUE 08

What is the return over several days?

For the full period, compare the last close with the first close. Daily percentage changes must compound because each starts from a different price.

Use the same six MSFT closes, August 14-21. Let R be the full-period price return. The first two growth factors illustrate why intermediate prices cancel.

$$(1 + r_1)(1 + r_2) = \frac{C_1}{C_0} \frac{C_2}{C_1} = \frac{C_2}{C_0}$$

Before running: On August 16, the daily price return is negative. Does that necessarily make the return since August 14 negative?

```
daily_return = close.pct_change(fill_method=None).iloc[1:]
cumulative_return = (1 + daily_return).cumprod() - 1
endpoint_return = close.iloc[-1] / close.iloc[0] - 1
log_sum = np.log1p(daily_return).sum()
result = pd.DataFrame({
    "Date": daily_return.index,
    "Daily return": daily_return.map(lambda r: f"{r:.2%}"),
    "Since Aug 14": cumulative_return.map(lambda r: f"{r:.2%}"),
})
result
```

Date	Daily return	Since Aug 14
2024-08-15	1.00%	1.00%
2024-08-16	-0.61%	0.39%
2024-08-19	0.73%	1.12%
2024-08-20	0.78%	1.90%
2024-08-21	-0.16%	1.75%

August 16's daily return is -0.61%, but the gain since August 14 remains 0.39%. The last cumulative value is 1.75%.

Using only the first and last close gives 1.7464%, equal to the compounded result before rounding. Converting the sum of log returns with $\exp(\log_sum) - 1$ also gives 1.7464%. These are three ways to describe the same price ratio, not three different investment outcomes.

Discuss

1. Use just August 15 and 16's daily returns to recover the return since August 14 at August 16. Explain why adding those two percentages is not the exact calculation.
2. If one day's return is omitted, why is equality with the endpoint return no longer assured? Identify the missing price ratio.

Sources: [7] Zivot: Return Calculations; [10] Yahoo Finance: MSFT historical data. See the linked references at the end.

ISSUE 09

Which average return should we use?

Use the arithmetic mean to describe the average observed one-period return. Use the geometric mean to find a constant per-period return that compounds to the same ending value.

There are $n = 5$ close-to-close returns in the same August 14-21 sample. The bar over r denotes their arithmetic mean; g is their geometric mean. R is the full-period return from Issue 8. All rates are decimals in the formulas.

$$\bar{r} = \frac{r_1 + \dots + r_n}{n}, \quad (1 + g)^n = 1 + R$$

To find g , take the n th root of the full-period growth factor and subtract one. This asks for an equivalent constant rate, not a forecast of tomorrow's return.

$$g = (1 + R)^{1/n} - 1$$

Before running: Which of these averages, repeated for five intervals, must reproduce the observed full-period return?

```
n = len(daily_return)
arithmetic_mean = daily_return.mean()
geometric_mean = (1 + endpoint_return)**(1 / n) - 1
result = pd.DataFrame({
    "Average": ["Arithmetic", "Geometric"],
    "Per interval": [f"{arithmetic_mean:.4%}", f"{geometric_mean:.4%}"],
})
result
```

Average	Per interval
Arithmetic	0.3488%
Geometric	0.3469%

The averages are 0.3488% and 0.3469% per close-to-close interval. Compounding the geometric mean five times gives 1.7464%, the observed full-period price return.

Here 'per interval' means one adjacent pair of trading-date closes. A Friday-to-Monday interval includes the weekend; it is not one calendar day.

Discuss

1. Calculate $(1 + \text{geometric_mean})^5 - 1$ and compare it with Issue 8. What exactly does their agreement establish?
2. A report calls the arithmetic mean the 'constant daily growth rate.' Rewrite that statement using the appropriate average, without claiming a future return.

Sources: [7] Zivot: Return Calculations. See the linked references at the end.

ISSUE 10

Is an absent date a missing price?

No row on a non-trading day is different from a missing price on a trading day. Do not turn either situation into a measured zero return.

MSFT's August 16 and 19 observations are Friday and Monday. August 17-18 have no regular-session daily bars. Nasdaq's regular session was 09:30-16:00 New York time on these trading dates; detailed time-zone conversion is not needed for this calculation.

Before running: Which two calendar dates should have no regular-session row? Does their absence mean the download lost a trading day's price?

```
result = pd.DataFrame({
    "Date": ["2024-08-16", "2024-08-17", "2024-08-18", "2024-08-19"],
    "Day": ["Friday", "Saturday", "Sunday", "Monday"],
})
result["Row returned"] = result["Date"].isin(close.index).map({True: "Yes", False: "No"})
result
```

Date	Day	Row returned
2024-08-16	Friday	Yes
2024-08-17	Saturday	No
2024-08-18	Sunday	No
2024-08-19	Monday	Yes

The Saturday and Sunday rows are absent from the source, not deleted for this example. The return ending on Monday uses Friday's close. Its interval spans three calendar days, but only one adjacent close-to-close step.

Before running: Can all six prices be present while the first return is still missing?

```
returns_with_first = close.pct_change(fill_method=None)
print("Missing Close values:", int(close.isna().sum()))
print("Missing return values:", int(returns_with_first.isna().sum()))
print("First return date:", returns_with_first.index[0])
```

```
Missing Close values: 0
Missing return values: 1
First return date: 2024-08-14
```

There are no missing Close values and one missing return. The return ending August 14 has no starting close inside this sample. It is unknown, not zero; use an earlier observation if you need that return.

Discuss

1. Before filling a gap on a weekday, what would you check in the exchange calendar and source request? Why is a weekday alone not proof that the market was open?
2. Why does filling the first missing return with zero change the average's meaning? State which missing starting observation would resolve the problem instead.

Sources: [1] yfinance: historical-data parameters; [8] Nasdaq: market hours and calendar. See the linked references at the end.

ISSUE 11

Should we delete a very large return?

A large price change deserves investigation, not automatic deletion. Netflix provides a real event that the small Microsoft sample does not contain.

Netflix released its Q1 2022 shareholder letter on April 19. Calculate the closing-price change from April 19 to April 20; then separate the observed change from an explanation of its cause.

Before running: Would a rule that deletes every daily return below -20% remove this observation? Would that rule, by itself, establish that the price is wrong?

```
netflix = yf.Ticker("NFLX").history(
    start="2022-04-19", end="2022-04-21",
    auto_adjust=False, actions=True, keepna=True,
)
nflx_close = netflix["Close"]
assert len(nflx_close) == 2 and nflx_close.notna().all()
nflx_return = nflx_close.iloc[-1] / nflx_close.iloc[0] - 1
print(f"NFLX price return, Apr 19-20: {nflx_return:.2%}")
print("Split events in request:", int(netflix["Stock Splits"].ne(0).sum()))
```

```
NFLX price return, Apr 19-20: -35.12%
Split events in request: 0
```

The source gives a -35.12% price return and no split event in the request. The shareholder letter documents a real company announcement. It does not establish that one news item explains the entire price change, or independently verify both price observations.

Before correcting a value

Confirm the security, dates, price field and share basis; compare the prices with another reliable source and inspect company events. Correct a documented error, not an uncomfortable result. A rule can flag an observation for review without removing it.

Discuss

1. Would you delete this row solely because its return is below -20%? Identify the evidence needed to justify changing the recorded price.
2. Why can deleting this loss make a return summary misleading? What does the dated shareholder letter support, and what causal claim does it not establish?

Sources: [1] yfinance: historical-data parameters; [9] Netflix: Q1 2022 shareholder letter. See the linked references at the end.

REFERENCES & REPRODUCTION

Sources and calculation notes

The tables contain selected event facts and calculated summaries, not a redistribution of the downloaded market-data files. The FX rate is transcribed from the Federal Reserve's dated release, not invented. See each issue for its inputs and assumptions.

- [1] [yfinance: historical-data parameters](#)
- [2] [Microsoft: dividend announced June 12, 2024](#)
- [3] [Investor.gov: ex-dividend dates](#)
- [4] [Apple: four-for-one split announcement](#)
- [5] [Federal Reserve: H.10, August 19, 2024](#)
- [6] [FRED: DEXTAUS units and observation time](#)
- [7] [Zivot: Return Calculations](#)
- [8] [Nasdaq: market hours and calendar](#)
- [9] [Netflix: Q1 2022 shareholder letter](#)
- [10] [Yahoo Finance: MSFT historical data](#)

Reproduce the examples

Run Issue 1 first, then continue in order. Each last table expression displays in a Python notebook; in a plain Python script, use `print(result)`. Install dependencies only if they are missing: `python -m pip install numpy pandas yfinance`.

Executed 2026-09-14 (UTC). Python 3.12.9; numpy 2.2.6; pandas 2.2.3; yfinance 1.2.0. The printed summaries and tables were generated by those displayed code blocks. Full-precision values are used before rounding for display.

Historical provider observations can be revised. Internet access and provider availability are required to repeat downloads. Yahoo's historical-page web preview was unavailable during this build; downloads through yfinance succeeded. Price-field conventions follow the course's prior source check; the exact adjustment algorithm is not re-derived here.

This short sample teaches calculation, not expected returns or a trading recommendation. Reading the log-return identity does not establish a distributional model. Calendar checks do not establish that every observed price is correct.